arXiv:2603.04208v1 [cs.RO] 4 Mar 2026

GSeg3D: A High-Precision Grid-Based Algorithm for Safety-Critical Ground Segmentation in LiDAR Point Clouds

Muhammad Haider Khan Lodhi*
German Research Center for Artificial Intelligence (DFKI)
Robotics Innovation Center
Bremen, Germany
Email: Haider_Khan.Lodhi@dfki.de
ORCID: 0009-0008-1199-3489
*Corresponding author

Christoph Hertzberg
German Research Center for Artificial Intelligence (DFKI)
Robotics Innovation Center
Bremen, Germany
Email: Christoph.Hertzberg@dfki.de
ORCID: 0009-0005-6467-0286

***Abstract*—Ground segmentation in point cloud data is the process of separating ground points from non-ground points. This task is fundamental for perception in autonomous driving and robotics, where safety and reliable operation depend on the precise detection of obstacles and navigable surfaces. Existing methods often fall short of the high precision required in safety-critical environments, leading to false detections that can compromise decision-making. In this work, we present a ground segmentation approach designed to deliver consistently high precision, supporting the stringent requirements of autonomous vehicles and robotic systems operating in real-world, safety-critical scenarios.**

***Index Terms*—Ground Segmentation, Grid-Based Segmentation, LiDAR, Robotics, Autonomous Driving.**

I. Introduction

The perception system of an autonomous vehicle relies on multiple sensors to capture critical features of the surrounding environment. Among these, LiDAR has become a core technology, providing dense, high-resolution point clouds that enable detailed spatial understanding [1]. A single LiDAR scan can contain over a million points, capturing both ground and non-ground surfaces within the region of interest. However, not all points are equally relevant for downstream perception tasks.

For example, object and obstacle detection algorithms often misclassify ground points as obstacles, leading to false positives that can compromise safety and increase computational load [2]. Accurate ground segmentation is essential for reliable perception, traversability analysis, navigation, and map generation in autonomous systems [3]–[7].

This work has been carried out within the project Robdekon2 (grant number: 13N16537), funded by the German Federal Ministry of Education and Research (BMBF). All code will be published after acceptance.

© 2025 IEEE. Personal use of this material is permitted.

This is the author's accepted manuscript of the paper published in: *2025 7th International Conference on Robotics and Computer Vision (ICRCV)*.
The final published version is available at: https://ieeexplore.ieee.org/document/11349133

In this work, we address the need for high-precision ground segmentation in LiDAR point clouds which is a critical requirement for safety-critical applications in autonomous driving and robotics. We present GSeg3D, a robust hybrid algorithm designed to maximize precision and reduce false positives, enabling safer and more effective decision-making in downstream perception modules. To validate its effectiveness, we rigorously evaluate GSeg3D on the challenging SemanticKITTI benchmark, where it demonstrates strong performance for ground extraction in autonomous systems.

II. Related Work

Ground segmentation methods in autonomous driving and robotics typically fall into two categories: traditional techniques and deep learning-based approaches [1].

Traditional methods primarily rely on geometric and statistical modeling of point cloud data. These methods typically utilize occupancy grids or elevation maps to spatially discretize the environment for efficient ground identification [8], [9].

Other traditional methods focus on modeling local surface characteristics using Gaussian process regression, plane fitting, and line extraction techniques. Himmelsbach et al. [10] proposed a line extraction approach, achieving high precision up to 98.5% in well-structured urban environments, but struggled in complex terrains. Gaussian Process Regression (GPR)

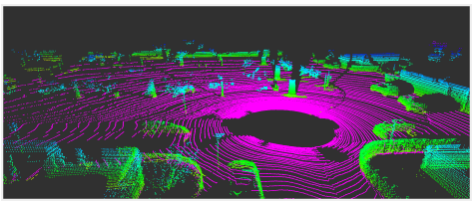

Fig. 1: GSeg3D: The point cloud is segmented into ground points (magenta) and non-ground points, which are color-coded according to their height to visualize vertical structures.

methods [11] have reported precision around 94.8%, showing robust performance in urban and semi-structured settings. Similarly, CascadedSeg [12] combined plane fitting and elevation thresholds, achieving precision of approximately 97% in structured urban datasets like SemanticKITTI. Patchwork [13] and its successor Patchwork++ [14] are patch-based ground segmentation algorithms achieving high precision. Patchwork typically reaches 92–94%, while Patchwork++ improves further to 93–95%, due to enhanced local modeling and refined patch updates.

Region-growing approaches and clustering techniques further refine segmentation by exploiting spatial continuity. Zermas et al. [15] employed a region-growing method based on planar characteristics, reporting precision of approximately 95% but with significant drops in cluttered or vegetated environments. Methods such as R-GPF [16], a refinement of GPF using multi-resolution grids, enhanced the recall but faced challenges maintaining precision, dropping to 84% in complex environments.

Advanced approaches using Conditional Random Field (CRF) and Markov Random Field (MRF) incorporate spatial coherence explicitly. Huang et al. [17] demonstrated an MRF-based approach with precision values around 93%, though the computational complexity was notably higher.

Deep learning approaches have gained traction due to their high flexibility and end-to-end training capabilities. Convolutional Neural Networks (CNN) initially adapted from object detection tasks have been successfully repurposed for ground segmentation in point cloud data. PointNet [18] and PointNet++ [19] offered unified frameworks achieving precision exceeding 96% in various segmentation tasks, although they require extensive training data.

PointPillars [20], designed for real-time detection and segmentation in autonomous driving scenarios, reported precision rates near 97% on SemanticKITTI datasets. SectorGSnet [21] introduced an angular-sector-based neural network model specifically for ground segmentation, achieving precision up to 98%, indicating strong potential for real-time autonomous driving applications.

Graph-based approaches such as GATA [22] and GndNet [23] employ attention mechanisms and graph neural networks. GndNet achieved precision scores around 96%, demonstrating robustness in various urban datasets. Meanwhile, JCP [24] combined CNN and point-based representations to handle complex terrains, achieving precision around 95%, though computational overhead remains a challenge.

Overall, both traditional and learning-based methods achieve high precision in structured environments but frequently exhibit performance degradation in unstructured, cluttered, or vegetation-rich terrains. This emphasizes the continued importance of robust geometric and statistical approaches that ensure high precision for safety-critical autonomous systems.

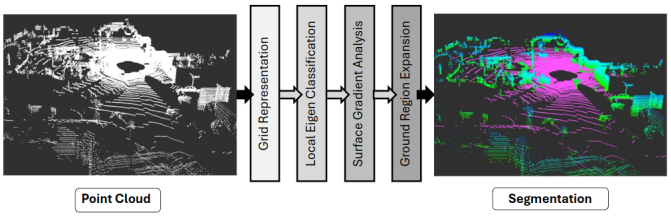

Fig. 2: Each phase consists of a four-step process. The figure displays the raw point cloud (in white), the detected ground points (highlighted in magenta), and the non-ground points, which are color-coded according to their height along the positive z-axis.

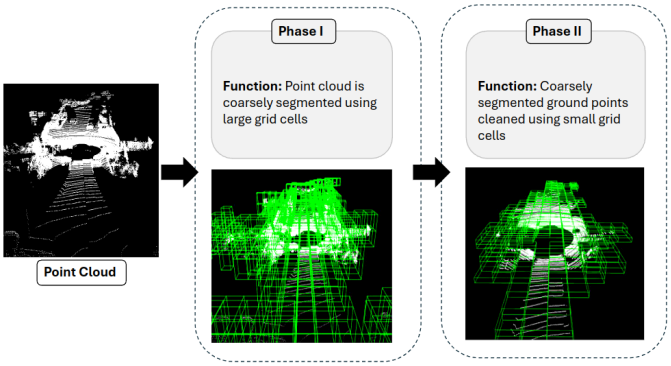

Fig. 3: Phase I uses grid cells with large height to capture tall structures as non-ground points. The coarse ground points from Phase I are subsequently processed in Phase II using grid cells with small height.

III. Proposed Solution

GSeg3D takes a single point cloud as input and outputs ground and non-ground points. The segmentation is performed in two distinct phases (Fig. 3).

In the first phase, a larger grid cell height is used to capture as many non-ground structures as possible. This approach ensures that even elevated objects are quickly marked as non-ground, but it can occasionally misclassify some ground points, especially if the ground is partially covered by objects.

To address this, GSeg3D employs a second phase that uses a much smaller grid cell height. This phase refines the initial segmentation by re-examining points at finer vertical resolution, allowing the method to correct for false positives and false negatives from the first stage. Specifically, points initially misclassified are re-evaluated based on their local neighborhood and vertical structure, as illustrated in Fig. 4.

This dual-phase design enables GSeg3D to achieve both high precision and high recall. By combining a coarse initial filtering with a targeted refinement, GSeg3D accurately distinguishes ground and non-ground points, which is critical for downstream tasks that require reliable terrain awareness.

Table I summarizes the differences between the two phases. The next sections describe each step in more detail.

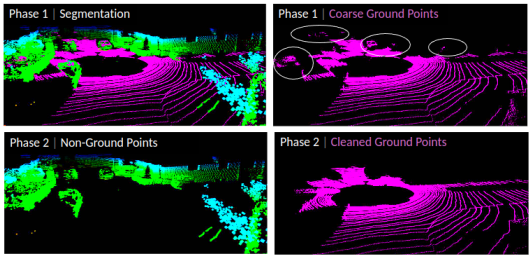

Fig. 4: Correction of over-segmentation of ground points based on dual-phase segmentation.

TABLE I: Processing steps in 1st and 2nd segmentation phase.

Step	**First Phase**	**Second Phase**
Grid Representation	Large cell height	Small cell height
Local Eigen Classification	Identical	Identical
Surface Gradient Analysis	Identical	Identical
Ground Region Expansion	Neighbor expansion	Neighbor expansion and additional point-wise check

A. Grid Representation

Each point in the input point cloud is assigned to a cell in a regularly spaced 3D grid. This discretization enables efficient modeling and detection of free, occupied, and ambiguous regions within the scene. The cell index for a given point is computed as:

$$cell_{\{x,y,z\}} = \left\lfloor \frac{point_{\{x,y,z\}}}{cellsize_{\{x,y,z\}}} \right\rfloor \tag{1}$$

where $cellsize$ defines the spatial resolution of the grid in each dimension.

This grid-based representation forms the basis for subsequent segmentation steps. It enables fast neighbor queries and local geometric analysis, which are critical for distinguishing ground from non-ground points in large-scale point clouds.

B. Local Eigen Classification

We perform local eigen classification to characterize the spatial arrangement of points within each grid cell. For every cell,

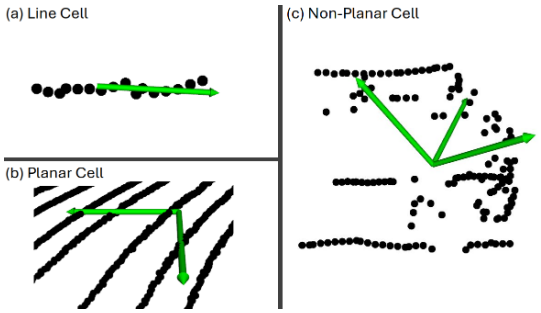

Fig. 5: Points (black) assigned to a grid cell with dominant eigenvectors (green) are shown. The cell is classified as Line (a), Planar (b), or Non-Planar (c) based on the local eigen classification.

we first compute the *covariance matrix* of the 3D coordinates of its points:

$$\mathbf{C} = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T \tag{2}$$

where $\mathbf{p}_i \in \mathbb{R}^3$ denotes the coordinates of the i-th point, $\bar{\mathbf{p}}$ is the centroid of all points in the cell, and N is the number of points.

The eigenvalues λ_1, λ_2, and λ_3 of $\mathbf{C}$ describe how the points are distributed along the principal axes of variation:

- $\lambda_1 \gg \lambda_2, \lambda_3$: points are aligned mainly in one direction,
- $\lambda_1 \approx \lambda_2 \gg \lambda_3$: points lie predominantly on a plane,
- $\lambda_1 \approx \lambda_2 \approx \lambda_3$: points are scattered in all directions.

Let $\lambda_1 > \lambda_2 > \lambda_3$ denote the eigenvalues in decreasing order. We then compute the ratio:

$$\text{Ratio} = \frac{\lambda_1}{\lambda_1+\lambda_2+\lambda_3} \tag{3}$$

This ratio reveals whether the points are primarily aligned in a single direction (line), distributed along a surface (plane), or lack any clear geometric preference (non-planar).

1) Line Cell: A cell is categorized as a Line Cell if λ_1 is much larger than λ_2 and λ_3, indicating strong alignment of points along a single direction. For such cells, we compute the angle between the principal eigenvector (corresponding to λ_1, shown in green in Fig. 5a) and the z-axis.

$$\text{CellType} = \begin{cases} \text{Obstacle} & \text{if angle} \geq 90^\circ - \text{Threshold} \\ \text{Tentative Ground} & \text{if angle} < 90^\circ - \text{Threshold} \end{cases} \tag{4}$$

Here, $Threshold$ is the maximum slope considered for ground points. Cells with an orientation below this slope threshold are provisionally marked as tentative ground, to be further verified in the region growth stage (see Section III-D).

2) Planar Cell: Cells with two dominant eigenvalues and a third, much smaller eigenvalue are classified as Planar Cells. This pattern indicates the points are distributed in a surface-like manner. However, because such distributions may arise from both ground surfaces and flat obstacle surfaces (e.g., roofs), Planar Cells are not directly labeled as ground without additional checks.

3) Non-planar Cell: If the eigenvalues are of similar magnitude, the cell is classified as Non-Planar. This suggests the points within the cell are scattered in space with no strong alignment, typical of clutter or irregular features.

C. Surface Gradient Analysis

For cells classified as planar in the local eigen classification, a robust plane fitting procedure is applied to accurately characterize the local surface geometry. We employ RANSAC-based plane fitting, which separates the points in each planar cell into two categories: *inliers*, which closely fit the estimated plane within a specified threshold, and *outliers*, which lie farther away.

Once a plane is fitted, we calculate its normal vector and determine the slope angle with respect to the horizontal plane

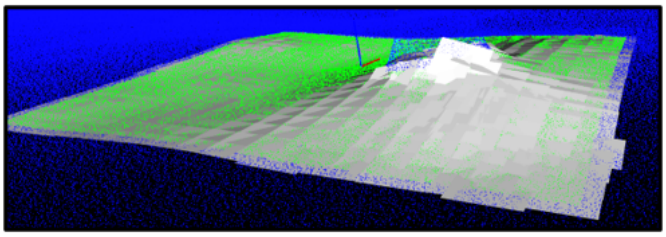

Fig. 6: To estimate the local surface slope, a plane model is fitted within each grid cell, allowing for a detailed and localized analysis of the terrain's inclination.

(the xy-plane), as shown in Fig. 6. This slope angle serves as the principal criterion for further cell classification:

$$\text{CellType} = \begin{cases} \text{Tentative Ground}, & \text{if angle} \leq \text{Threshold} \\ \text{Non-Ground}, & \text{if angle} > \text{Threshold} \end{cases} \tag{5}$$

Here, *Threshold* denotes the maximum slope considered compatible with typical ground surfaces. Cells whose fitted plane slope is less than or equal to this threshold are labeled as *tentative ground*; those with steeper slopes are classified as *non-ground*. The tentative ground cells determined in this manner undergo further verification in the subsequent stages of the algorithm to ensure accurate segmentation.

This step is crucial for distinguishing between relatively flat ground regions and sloped surfaces that are unlikely to be traversable, improving the precision of the ground/non-ground segmentation.

D. Ground Region Expansion

A well-known challenge in grid-based ground segmentation approaches is the dependency of region expansion on grid cell size. When the cell size is smaller than the gap between consecutive scan lines (as is common in sparse or non-uniform point clouds), traditional grid-based neighbor expansion fails. This leads to disconnected ground regions and poor recall, since expansion cannot bridge the gaps between scan lines. As a result, grid-based methods are often forced to use larger cell sizes at the expense of spatial precision and fine-grained segmentation.

GSeg3D addresses this limitation by adopting a KD-tree–based expansion strategy. All centroids of candidate ground cells are indexed using a KD-tree, enabling efficient and radius-based neighborhood search in three dimensions. This allows for region expansion even when cells are not directly adjacent in the grid, but are spatially proximate in the point cloud. As a result, much smaller grid cell sizes can be used without sacrificing connectivity or recall.

Combined with local eigen classification, which ensures that each cell's geometric structure is accurately characterized, this approach enables high-precision and high-recall ground segmentation at finer spatial resolutions.

The selection of the initial seed cell for region expansion is critical: a poor seed can adversely affect segmentation quality. To ensure a reliable and robust start, GSeg3D injects synthetic points directly below the robot, based on the `robotRadius` and `distToGround` parameters. The cell corresponding to the position directly underneath the robot (i.e., the synthetic seed cell) is always used as the starting point for expansion, guaranteeing that the initial ground seed is valid, even in the presence of sensor gaps or occlusions. These synthetic points are removed prior to final metric calculation and output.

The expansion proceeds as follows:

- The centroids of all tentative ground cells are indexed in a KD-tree for fast neighbor queries.
- Expansion is initialized from the seed cell directly below the robot.
- For each dequeued candidate cell, all neighboring centroids within a specified radius are queried via the KD-tree.
- In Phase II (refinement), only neighbors within a height difference threshold are considered to maintain consistency.
- Valid neighboring cells are queued for further expansion if they are also classified as tentative ground.

After region expansion, each candidate cell undergoes further refinement using the following procedure, closely following the implemented logic:

1) **Extraction of Inliers and Outliers:** Each cell's points are divided into *ground inliers* (those close to the previously fitted plane) and *non-ground inliers* (those further away), based on the indices identified during the Surface Gradient Analysis.
2) **Bounding Box Sparsity Analysis:** For both sets of points (ground and non-ground inliers), a three-dimensional bounding box is computed and its sparsity is evaluated. The sparsity score, defined as the ratio of bounding box volume to the number of points, is categorized as "Low," "Medium," or "High." If both inlier groups receive the same sparsity label, the bounding box analysis is considered ambiguous and additional checks are triggered.
3) **Ambiguity Resolution via Local Neighborhood:** When bounding box scores are ambiguous, the algorithm compares the height of the centroid of the current cell's ground inliers with those of neighboring ground cells. If no valid ground neighbors are found, or if the cell centroid is significantly elevated (by more than $0.3\,m$) above the lowest neighbor, the cell is rejected as ground to prevent mislabeling of elevated structures.
4) **Floating Cell Rejection:** To avoid classifying floating structures as ground, the method checks for any occupied non-ground cell directly beneath the current cell in the vertical stack. If such a cell exists, the current cell is labeled as non-ground.
5) **Final Assignment:** Only cells passing all previous checks are assigned as ground, while those failing any step are classified as non-ground.

This multi-step refinement, combining geometric, density-

Algorithm 1 Ground Region Expansion

Config: r: search radius, τ: inlier threshold, phase
Input: Seed queue Q, tentative ground centroids C, grid cells $\mathcal{G}$
Output: Segmented point sets P_{ground}, $P_{\text{non-ground}}$
Build KD-tree for centroids C
while Q not empty **do**
    Dequeue cell i from Q; mark i as expanded
    if cell i is empty **then**
        continue
    end if
    // Perform radius search around centroid c_i within r:
    $N = neighbors(c_i, r)$
    for each $j \in N$ **do**
        if j expanded or $j \in Q$ **then**
            continue
        end if
        if $phase = 2$ and $|c_{i,z} - c_{j,z}| > \tau$ **then**
            continue
        end if
        Mark cell j as GROUND; enqueue j to Q
    end for
    Extract ground inliers and non-ground outliers in i
    if ground inliers are empty **then**
        continue
    end if
    Classify sparsity for inliers and outliers
    if scores equal **then**
        Compute $z_i = height(i)$, gather z_n of neighbors
        if neighbors empty **or** $|z_i - z_n| > \tau$ **then**
            Mark as non-ground; **continue**
        end if
        if non-ground cell exists below **then**
            Mark as non-ground; **continue**
        end if
    end if
    Add inliers to P_{ground}, outliers to $P_{\text{non-ground}}$
end while
return $P_{\text{ground}}, P_{\text{non-ground}}$

based, and local contextual analysis, ensures that only spatially coherent and physically plausible regions are segmented as ground. The resulting segmentation achieves robust, high-precision ground extraction suitable for safety-critical applications.

The KD-tree driven iterative expansion thus enables robust, contiguous ground region extraction across the point cloud, even at high grid resolutions. This method overcomes the fundamental connectivity limitations of traditional grid-based expansion and is especially effective for safety-critical applications where accurate and dense ground segmentation is required.

IV. Experiments

For comparison, we selected state-of-the-art ground segmentation algorithms as listed in Table II, using their default or recommended parameters from the respective publications or open-source implementations. For all experiments with GSeg3D, we used a grid size of $(1.5, 1.0, 1.5)$m in Phase I and $(1.5, 1.0, 0.2)$m in Phase II. The ground inlier threshold was set to 0.125m and the ground slope threshold to $30°$. The following configuration was used:

```
distToGround: 1.723
robotRadius: 2.7
cellSizeX: 1.5
cellSizeY: 1.0
cellSizeZ: 1.5 (Phase I), 0.2 (Phase II)
slopeThresholdDegrees: 30.0
groundInlierThreshold: 0.125
centroidSearchRadius: 5.0
```

A. Dataset

We evaluated all algorithms on the SemanticKITTI dataset [25], which consists of large-scale LiDAR point clouds acquired in urban environments. The data was collected using a car platform equipped with a Velodyne HDL-64E sensor mounted at a height of 1.72m. This dataset, recorded in 2013, provides high-resolution, annotated scans that serve as a standard benchmark for autonomous driving research.

B. Performance Metrics

To quantitatively evaluate segmentation performance, we use precision, recall, and F1-score as our primary metrics:

$$\text{Precision} = \frac{\text{NTP}}{\text{NTP+NFP}} \tag{6}$$

$$\text{Recall} = \frac{\text{NTP}}{\text{NTP+NFN}} \tag{7}$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision+Recall}} \tag{8}$$

where

- **NTP**: Number of True Positives (correctly predicted ground points),
- **NFP**: Number of False Positives (non-ground points incorrectly predicted as ground),
- **NFN**: Number of False Negatives (ground points incorrectly predicted as non-ground).

To provide a robust and comprehensive comparison, we evaluate each algorithm across multiple distance thresholds (from 10m to 100m, in 10m increments) for each dataset sequence. For every distance threshold, we compute the standard metrics of precision, recall, and F1-score.

For clarity and conciseness in reporting, we aggregate the results for each sequence by taking the arithmetic mean and standard deviation of the metric values across all tested distances. Thus, the final result for each algorithm and sequence is presented as "mean $\pm$ std", representing the average performance and its variability over the entire evaluation range.

This approach follows established practice in benchmarking and reflects both the typical performance and its stability

with respect to varying perception ranges. All quantitative results in this paper are based on these metrics, with ground-truth labels provided by the SemanticKITTI dataset. The F1-score provides a harmonic mean between precision and recall, offering a balanced measure of segmentation performance.

V. Results and Discussion

The performance of eight ground segmentation algorithms was evaluated on sequences 00 to 10 of the SemanticKITTI dataset, with detailed quantitative comparisons presented in Tables II, and III. Metrics reported include precision (Pr.), recall (Rc.), and F1-score, each accompanied by standard deviations (std).

A. Urban Environments (Sequences 00, 02, 05, 06, 07, 09)

Urban sequences typically involve structured roadways, sidewalks, and clutter from parked cars and street furniture. In these scenarios, *Linefit* consistently demonstrated the highest precision (often exceeding 98%) due to its linear-model assumptions aligning well with structured roads. However, *GSeg3D*, utilizing a localized grid-based spatial reasoning approach, closely matched this high precision (above 97%) while offering improved adaptability to complex urban geometries and irregularities such as curbs, ramps, and potholes.

Patchwork and *Patchwork++* maintained stable and high precision (approximately 93-96%), benefitting from their iterative patch-based refinement suited to urban contexts.

Regarding recall in urban environments, *Patchwork++* and *Patchwork* consistently achieved high recall values (typically above 90%), benefiting from their localized segmentation strategies that effectively capture ground points even in cluttered scenes. *GSeg3D*, while maintaining robust recall performance (generally between 85% and 90%), exhibited slightly lower values compared to the Patchwork-based methods. Nevertheless, its recall remains well-balanced and sufficiently high for reliable ground segmentation in urban scenarios. Conversely, methods such as *CGSeg* and *GPF*, which exhibited significantly lower recall (below 80% in some urban sequences), struggled in densely cluttered areas or scenes with abrupt terrain transitions.

In urban contexts, the F1-score revealed *GSeg3D* matched closely *Patchwork++* as a highly balanced solution (93%-95%). However, *GSeg3D's* high recall is due to consistently high precision values (above 97%), whereas *Patchwork++* has higher recalls (above 91%).

B. Suburban and Highway Environments (Sequences 01, 03, 04)

Suburban and highway sequences are characterized by open roads, moderate clutter, consistent, and predictable terrain. Here, methods like *Linefit* again excelled in precision due to the predominantly linear road geometry (precision $\sim$99%). However, *GSeg3D* still showed comparable performance (97-99%), leveraging its grid-based segmentation to maintain robust adaptability around road edges, lane transitions, and minor roadside variations.

Recall performance in these environments strongly favored algorithms with localized approaches such as *Patchwork++*, achieving recall values above 95%. *GSeg3D* similarly showed consistently strong recall (around 89-91%), demonstrating balanced detection capability essential for highway automation, where consistent ground segmentation ensures stable vehicle control.

In terms of F1-score, *Patchwork++* (around 94%) and *GSeg3D* (92-94%) were particularly robust, balancing precision and recall effectively, making them suitable candidates for suburban autonomous driving scenarios.

C. Complex and Vegetation-Rich Environments (Sequences 08, 10)

Complex terrains featuring dense vegetation, uneven ground, and irregular structures presented the greatest segmentation challenge. Here, global fitting methods such as *Linefit* and *Ransac* faced higher variability and performance degradation, particularly noticeable in their recall (below 80%).

In contrast, the localized, grid-based strategy of *GSeg3D* allowed it to adapt effectively, maintaining high precision (above 94%) and moderate recall (85-87%) despite increased environmental complexity. *R_GPF*, characterized by high recall (above 95%) yet lower precision (below 70%), proved valuable for exploratory tasks where capturing ground points takes precedence over precision.

Patchwork++ maintained balanced precision and recall, highlighting its suitability for general-purpose deployment across mixed or uncertain terrains. The stability in the F1-score (89-93%) demonstrated by *GSeg3D* coupled with high precision (above 94%) across these challenging sequences further confirmed its robustness for applications such as off-road robotics and outdoor mobile mapping.

D. Overall Performance Analysis

Table III summarizes the average performance metrics across all SemanticKITTI sequences.

Precision: *Linefit* achieved the highest average precision (97.9%) with the lowest variability (1.4%), making it highly reliable in structured environments. *GSeg3D* followed closely with high precision (96.6%) and low variability (2.7%), offering consistently strong performance across diverse and complex scenarios.

Recall: *R_GPF* achieved the highest recall (95.3%) but suffered from low precision. In contrast, *Patchwork++*, *Patchwork*, and *GSeg3D* offered balanced recall performance (93.7%, 93.0%, and 89.4%, respectively), with *GSeg3D* maintaining superior precision.

F1-score: *Patchwork++* (93.5%) and *GSeg3D* (92.8%) achieved the highest F1-scores, demonstrating balanced accuracy and completeness.

Runtime: *Linefit* was fastest (6.0 ms), followed by *GPF* and *Patchwork++*. While slower (48.1 ms), *GSeg3D* provides a solid trade-off between efficiency and segmentation quality.

Stability: *GSeg3D* exhibited stable precision (2.7% std dev), second only to *Linefit*, reinforcing its predictable performance.

TABLE II: Comparison of ground segmentation methods on SemanticKITTI (sequences 00–10). Results are reported as mean ± std (%).

Method	00			01			02			03		
	Pr.	Rc.	F1	Pr.	Rc.	F1	Pr.	Rc.	F1	Pr.	Rc.	F1
CGSeg	91.6±9.1	71.0±10.6	80.0	**97.2**±2.9	75.7±10.5	85.1	91.8±7.7	70.0±7.7	79.4	92.4±8.0	69.0±8.1	79.0
GPF	95.1±4.9	80.9±25.5	87.4	93.9±6.4	42.6±23.5	55.7	96.4±3.5	79.0±18.5	86.8	**97.5±2.8**	78.7±13.5	87.0
Linefit	**98.2±1.0**	81.1±10.0	88.8	**99.0±1.4**	74.6±9.6	84.9	**98.0±1.2**	82.5±5.9	89.5	**95.8**±4.1	79.3±8.4	86.6
Patchwork	92.2±3.5	93.3±3.9	92.7	95.6±3.5	89.8±5.2	92.6	92.1±3.9	93.2±**3.8**	**92.6**	89.4±7.1	**95.3±2.7**	92.2
Patchwork++	93.8±3.9	**93.9±3.3**	**93.9**	89.9±6.4	**97.4±2.8**	**93.5**	93.8±3.9	**93.6**±3.9	**93.7**	94.7±**2.9**	91.0±6.6	**92.8**
R_GPF	56.5±11.4	**96.2±1.7**	71.2	88.0±6.4	90.6±4.3	89.3	65.6±13.1	**96.4±1.7**	78.1	72.0±10.1	**96.8±1.3**	82.6
Ransac	87.9±15.6	93.7±12.4	90.7	96.9±**2.2**	**96.3±3.5**	**96.6**	90.9±13.8	90.9±8.9	90.9	92.8±7.1	90.3±8.8	91.5
GSeg3D	**97.1±2.1**	89.9±5.7	**93.4**	**97.2**±3.7	85.5±7.9	90.9	**97.0±2.2**	90.7±4.2	**93.7**	94.1±6.2	92.0±4.3	**93.0**

Method	04			05			06			07		
	Pr.	Rc.	F1	Pr.	Rc.	F1	Pr.	Rc.	F1	Pr.	Rc.	F1
CGSeg	98.2±2.5	73.7±4.2	84.2	86.1±12.9	68.4±12.4	76.3	98.2±2.3	75.9±4.6	85.6	89.6±10.7	72.4±11.1	80.1
GPF	95.9±4.6	76.1±25.9	84.8	93.4±4.1	88.8±9.1	91.0	97.4±2.2	78.9±17.0	87.1	94.3±3.6	89.5±17.0	91.8
Linefit	**99.5±0.6**	78.2±4.7	87.5	**96.7±1.6**	83.2±6.8	89.3	**99.3±0.4**	80.5±3.5	88.8	**97.2±1.3**	87.6±5.6	92.0
Patchwork	97.4±1.4	91.5±2.6	**94.4**	89.3±5.0	**94.2±2.8**	91.6	96.8±**1.3**	96.2±**1.3**	**96.5**	90.2±4.2	**95.5±2.5**	92.7
Patchwork++	90.2±3.5	**97.9±1.3**	93.9	**94.9**±2.8	91.1±4.4	**93.0**	96.4±1.8	**97.7±1.0**	**97.1**	**96.4**±2.6	91.9±3.4	**94.1**
R_GPF	83.0±9.0	93.4±1.6	87.9	57.7±15.0	**96.2±1.3**	72.1	82.2±7.2	95.6±**1.3**	88.4	55.3±13.3	**97.1±1.2**	70.4
Ransac	96.1±2.1	**96.9±1.4**	**96.5**	85.7±13.5	93.8±7.5	89.6	95.5±2.6	**97.3**±1.8	96.4	84.4±18.0	94.4±11.7	89.1
GSeg3D	**99.3±1.2**	89.5±2.8	94.1	**94.9±2.5**	89.2±4.3	**92.0**	**98.5±1.3**	89.4±3.8	93.6	96.1±**2.2**	90.9±3.4	**93.4**

Method	08			09			10		
	Pr.	Rc.	F1	Pr.	Rc.	F1	Pr.	Rc.	F1
CGSeg	90.3±13.9	67.1±13.5	77.0	94.8±5.9	71.0±8.0	81.2	82.7±20.8	59.2±17.5	69.0
GPF	96.9±3.7	76.0±21.3	85.1	96.2±5.7	69.0±25.2	80.2	91.7±7.2	66.5±23.2	76.9
Linefit	**98.7±1.0**	77.2±9.1	86.5	**98.0±1.3**	78.8±8.7	87.3	**97.0±1.4**	76.7±10.9	85.6
Patchwork	93.2±4.6	92.5±**4.6**	**92.8**	91.5±4.0	92.2±4.2	91.8	88.4±5.0	**89.5±7.3**	88.9
Patchwork++	93.3±4.1	**94.5±4.6**	**93.9**	92.1±4.4	**93.8±3.7**	**92.9**	91.0±5.1	87.5±8.9	**89.2**
R_GPF	64.8±18.8	**95.2±2.5**	77.1	68.8±12.7	**95.2±3.0**	79.8	53.0±19.2	**95.4±2.7**	68.1
Ransac	85.8±23.8	88.7±18.6	87.2	93.5±8.0	89.9±6.1	91.7	72.4±28.2	82.6±19.8	77.2
GSeg3D	**98.0±2.0**	86.9±7.4	92.1	**96.8±2.5**	88.8±4.8	**92.6**	**94.2±3.4**	85.0±8.3	**89.4**

TABLE III: Average performance (%) across all sequences and Runtime (Rt.) in milliseconds.

Method	Pr.	Rc.	F1	Rt.	Pr. Std	Rc. Std	Rt. Std
CGSeg	92.1	70.3	79.7	59.1	8.8	9.8	11.0
GPF	95.3	75.1	83.1	**11.4**	4.4	20.0	**0.7**
Linefit	**97.9**	80.0	87.9	**6.0**	**1.4**	7.6	**0.4**
Patchwork	92.4	93.0	92.6	15.7	4.0	**3.7**	1.0
Patchwork++	93.3	**93.7**	**93.5**	13.2	3.8	4.0	1.6
R_GPF	67.9	**95.3**	78.6	17.4	12.4	**2.0**	1.1
Ransac	89.3	92.3	90.7	53.6	12.3	9.1	10.3
GSeg3D	**96.6**	89.4	**92.8**	48.1	**2.7**	5.1	7.3

In summary, *GSeg3D* offers an effective balance of high precision, reliable recall, stable output, and reasonable runtime, making it well-suited for real-world, safety-critical applications.

E. Failure Case Analysis and Runtime Variability

Our evaluation also highlights the limitations and variability observed across different environments. In vegetation-rich or highly cluttered scenarios (e.g., Sequences 08 and 10), global fitting methods such as *Linefit* and *RANSAC* exhibited reduced recall, while recall-focused methods like *R_GPF* maintained high completeness at the cost of precision. *GSeg3D*, although robust, also showed moderate recall degradation in these conditions, reflecting the inherent difficulty of distinguishing ground points from low-lying vegetation or occluded regions.

Future work will involve combining geometric reasoning with semantic cues (e.g., vegetation class segmentation) to improve the discrimination between true ground and vegetation, thereby enhancing robustness in such environments.

Regarding runtime, results in Table III demonstrate that *Linefit* and *GPF* are the fastest, with very low variability (std < 1 ms). In contrast, iterative methods such as *Patchwork++* and the proposed *GSeg3D* exhibit higher runtimes (13–48 ms) with greater variability due to their grid-based reasoning and refinement stages. Nevertheless, *GSeg3D* achieves a favorable balance: while not the fastest, its runtime remains practical for real-time applications, and its higher variability is compensated by consistently strong segmentation quality.

Application Suitability: Summary analysis suggests:

- **GSeg3D:** Best for safety-critical applications (autonomous driving, robotics) due to consistently high precision, good recall, and low variability.
- **Patchwork++ / Patchwork:** General-purpose methods with balanced precision and recall, suitable for urban mapping and general navigation.
- **Linefit:** Ideal for structured environments needing very high precision with minimal variability (e.g., infrastructure inspection).
- **R_GPF:** Suited for exploratory or recall-focused tasks, where completeness matters more than precision.
- **Ransac:** Suitable for moderate-complexity scenarios and

prototyping, balancing performance with higher variability.

- **GPF:** Applicable to static or controlled environments where occasional inaccuracies are acceptable.
- **CGSeg:** Useful for precision-focused tasks like boundary refinement or post-processing where lower recall is acceptable.

VI. Conclusion

Overall, the results demonstrate that *GSeg3D* offers near-top precision with notably low standard deviation, ensuring stable and predictable performance. Compared to *Linefit*, which achieves slightly higher precision, *GSeg3D* maintains significantly better recall while preserving high precision, resulting in a more balanced segmentation performance.

Unlike methods biased toward either precision or recall, *GSeg3D* consistently combines precise ground classification with reliable recall across diverse environments. This combination of high precision, good recall, low variability, and reasonable runtime makes *GSeg3D* particularly suitable for safety-critical applications in autonomous driving and robotics, where both reliability and consistency are essential.

Nonetheless, challenges remain in vegetation-rich or occluded environments, where recall performance degrades due to the difficulty of separating ground points from low vegetation or partially visible surfaces. Future work will focus on addressing these cases by combining geometric reasoning with semantic segmentation cues (e.g., vegetation class detection), enabling more robust ground segmentation across highly complex outdoor scenarios.

VII. Acknowledgment

We greatly acknowledge the open source Ground Segmentation Benchmark [13], [14] which we used for benchmarking. PCL [26] and Nanoflann [27] were used for processing the point clouds.

References

[1] T. Gomes, D. Matias, A. Campos, L. Cunha, and R. Roriz, "A survey on ground segmentation methods for automotive lidar sensors," *Sensors*, vol. 23, no. 2, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/2/601

[2] E. Firkat, F. An, B. Peng, J. Zhang, T. Mijit, A. Ahat, J. Zhu, and A. Hamdulla, "FGSeg: Field-ground segmentation for agricultural robot based on LiDAR," *Computers and Electronics in Agriculture*, vol. 211, p. 107965, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0168169923003538

[3] H. Lim, M. Oh, S. Lee, S. Ahn, and H. Myung, "Similar but different: A survey of ground segmentation and traversability estimation for terrestrial robots," *International Journal of Control, Automation and Systems*, vol. 22, no. 2, pp. 347–359, 2024.

[4] T.-C. Tsai and C.-C. Peng, "Ground segmentation based point cloud feature extraction for 3d lidar slam enhancement," *Measurement*, vol. 236, p. 114890, 2024.

[5] M. Arora, L. Wiesmann, X. Chen, and C. Stachniss, "Static map generation from 3d LiDAR point clouds exploiting ground segmentation," *Robotics and Autonomous Systems*, vol. 159, p. 104287, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889022001762

[6] V. Jiménez, J. Godoy, A. Artuñedo, and J. Villagra, "Ground segmentation algorithm for sloped terrain and sparse LiDAR point cloud," *IEEE Access*, vol. 9, pp. 132 914–132 927, 2021.

[7] H. Lim, B. Kim, D. Kim, E. M. Lee, and H. Myung, "Quatro++: Robust global registration exploiting ground segmentation for loop closing in lidar slam," *The International Journal of Robotics Research*, vol. 43, no. 5, pp. 685–715, 2024. [Online]. Available: https://doi.org/10.1177/02783649231207654

[8] B. Douillard, J. Underwood, N. Melkumyan, S. Singh, S. Vasudevan, C. Brunner, and A. Quadros, "Hybrid elevation maps: 3d surface models for segmentation," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010, pp. 1532–1538.

[9] B. Anand, M. Senapati, V. Barsaiyan, and P. Rajalakshmi, "Lidar-ins/gnss-based real-time ground removal, segmentation, and georeferencing framework for smart transportation," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–11, 2021.

[10] M. Himmelsbach, F. v. Hundelshausen, and H.-J. Wuensche, "Fast segmentation of 3d point clouds for ground vehicles," in *2010 IEEE Intelligent Vehicles Symposium*, 2010, pp. 560–565.

[11] T. Chen, B. Dai, R. Wang *et al.*, "Gaussian-process-based real-time ground segmentation for autonomous land vehicles[j]," in *Journal of Intelligent and Robotic Systems*, 2014, pp. 76(3–4):563–582.

[12] P. Narksri, E. Takeuchi, Y. Ninomiya, Y. Morales, N. Akai, and N. Kawaguchi, "A slope-robust cascaded ground segmentation in 3d point cloud for autonomous vehicles," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 497–504.

[13] H. Lim, M. Oh, and H. Myung, "Patchwork: Concentric zone-based region-wise ground segmentation with ground likelihood estimation using a 3d lidar sensor," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 6458–6465, 2021.

[14] S. Lee, H. Lim, and H. Myung, "Patchwork++: Fast and robust ground segmentation solving partial under-segmentation using 3d point cloud," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 13 276–13 283.

[15] D. Zermas, I. Izzat, and N. Papanikolopoulos, "Fast segmentation of 3d point clouds: A paradigm on lidar data for autonomous vehicle applications," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 5067–5073.

[16] H. Lim, S. Hwang, and H. Myung, "Erasor: Egocentric ratio of pseudo occupancy-based dynamic object removal for static 3d point cloud map building," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2272–2279, 2021.

[17] W. Huang, H. Liang, L. Lin, Z. Wang, S. Wang, B. Yu, and R. Niu, "A fast point cloud ground segmentation approach based on coarse-to-fine markov random field," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 7, pp. 7841–7854, 2022.

[18] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.

[19] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," *Advances in neural information processing systems*, vol. 30, 2017.

[20] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 12 697–12 705.

[21] D. He, F. Abid, Y.-M. Kim, and J.-H. Kim, "Sectorgsnet: Sector learning for efficient ground segmentation of outdoor lidar point clouds," *IEEE Access*, vol. 10, pp. 11 938–11 946, 2022.

[22] I. d. Pino, A. Santamaria-Navarro, A. Garrell Zulueta, F. Torres, and J. Andrade-Cetto, "Probabilistic graph-based real-time ground segmentation for urban robotics," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 5, pp. 4989–5002, 2024.

[23] A. Paigwar, Ö. Erkent, D. Sierra-Gonzalez, and C. Laugier, "GndNet: Fast ground plane estimation and point cloud segmentation for autonomous vehicles," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 2150–2156.

[24] Z. Shen, H. Liang, L. Lin, Z. Wang, W. Huang, and J. Yu, "Fast ground segmentation for 3d lidar point cloud based on jump-convolution-process," *Remote Sensing*, vol. 13, no. 16, 2021. [Online]. Available: https://www.mdpi.com/2072-4292/13/16/3239

[25] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *International Journal of Robotics Research (IJRR)*, 2013.

[26] R. B. Rusu and S. Cousins, “3d is here: Point cloud library (pcl),” in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 1–4.

[27] J. L. Blanco and P. K. Rai, “nanoflann: a c++ header-only fork of flann, a library for nearest neighbor (nn) with kd-trees,” *GitHub Repository*, 2014.